

InstanciaIris: Pipeline de Machine Learning com Rastreamento de Experimentos em MLflow e API de Predição

Diego Cordeiro de Oliveira¹

¹Departamento de Computação – Universidade Federal do Piauí (UFPI)
Teresina – PI – Brasil

diego.cordeiro@ifpi.edu.br

Abstract. *This technical report presents the instantiation and the end-to-end execution of the InstanciaIris project, a machine learning workflow built on the FabricaIA template. The goal was to establish a reproducible pipeline for the Iris dataset, covering preprocessing (label encoding and feature standardization), training of a Random Forest classifier with experiment tracking in MLflow, model persistence, and the exposure of a REST prediction endpoint via FastAPI. The notebook was executed without errors: the Random Forest model (100 estimators, maximum depth 10) achieved an accuracy of 0.9333 and an F1 score of 0.90, being registered in the MLflow experiment testprojeto_experiments. The prediction API answered a test request returning a class and the associated probabilities. A known limitation is highlighted: the /predict endpoint does not reapply the feature scaler; as future work, persisting the preprocessing pipeline and applying it during inference is recommended.*

Resumo. *Este relatório técnico apresenta a instanciação e a execução ponta a ponta do projeto InstanciaIris, um fluxo de aprendizado de máquina construído sobre o template FabricaIA. O objetivo foi estabelecer um pipeline reproduzível para o dataset Iris, abrangendo pré-processamento (codificação do rótulo e padronização das características), treinamento de um classificador Random Forest com rastreamento de experimentos em MLflow, persistência do modelo e exposição de um endpoint de predição REST via FastAPI. O notebook foi executado sem erros: o modelo Random Forest (100 estimadores, profundidade máxima 10) alcançou acurácia de 0,9333 e F1 de 0,90, sendo registrado no experimento testprojeto_experiments do MLflow. A API de predição respondeu a uma requisição de teste com a classe e as probabilidades associadas. Destaca-se a limitação conhecida: o endpoint /predict não reaplica o escalador de características; como trabalho futuro, sugere-se persistir o pipeline de pré-processamento e aplicá-lo na inferência.*

1. Introdução

Projetos de aprendizado de máquina não se encerram no treinamento de um modelo: a reprodutibilidade, o rastreamento de experimentos e a disponibilização do modelo para inferência são etapas igualmente relevantes para consolidar uma solução confiável. Nesse contexto, o uso de templates e de ferramentas de experimentação contribui para padronizar o fluxo de trabalho e reduzir o risco de perda de informação sobre versões, hiperparâmetros e métricas.

O dataset Iris, proposto por Fisher, é um benchmark clássico de classificação supervisionada, com 150 amostras e três classes de espécies de flor, sendo adequado para validar pipelines de pré-processamento, treinamento e avaliação.

Diante disso, este relatório documenta a instanciação e a execução do projeto InstanciaIris, com os seguintes objetivos: (i) instanciar um projeto de aprendizado de máquina a partir do template FabricaIA; (ii) construir um pipeline de pré-processamento e treinamento com rastreamento de experimentos em MLflow; (iii) expor um modelo treinado por meio de uma API REST com FastAPI; e (iv) validar o funcionamento ponta a ponta por meio de testes automatizados e de uma chamada de predição real.

2. Fundamentação Teórica

O aprendizado de máquina supervisionado busca aprender uma função que mapeia características de entrada para um rótulo de saída a partir de exemplos rotulados. O Random Forest é um método de conjunto (ensemble) baseado em árvores de decisão, no qual múltiplas árvores são construídas sobre subamostras aleatórias dos dados e de suas características, e a predição final é obtida por votação majoritária [Breiman 2001, Dietterich 2000]. A implementação utilizada neste trabalho é a fornecida pela biblioteca scikit-learn [Pedregosa et al. 2011].

O dataset Iris [Fisher 1936] contém 150 amostras organizadas em três classes (setosa, versicolor e virginica), caracterizadas por quatro medidas morfológicas: comprimento e largura da sépala e comprimento e largura da pétala.

Em tarefas de classificação, o pré-processamento é tipicamente composto pela codificação de variáveis categóricas e pela padronização de características numéricas. A codificação de rótulos (LabelEncoder) transforma as classes em valores inteiros, enquanto a padronização (StandardScaler) [Pedregosa et al. 2011] ajusta cada característica para média zero e desvio padrão unitário, evitando que a diferença de escala influencie o modelo.

O rastreamento de experimentos é fundamental para a reprodutibilidade em aprendizado de máquina. O MLflow [Zaharia et al. 2018] permite registrar experimentos, parâmetros, métricas e artefatos de modelos, organizados em execuções (runs) agrupadas em experimentos. Neste trabalho, o MLflow foi configurado com backend SQLite local (sqlite:///mlflow.db).

Para a avaliação de modelos de classificação, métricas como acurácia, precisão, recall e F1-score, derivadas da matriz de confusão, são amplamente utilizadas [Sokolova and Lapalme 2009]. A divisão dos dados em conjuntos de treino e teste é uma prática padrão de validação, e a estratificação é comumente empregada para preservar a distribuição das classes [Kohavi 1995]. Na busca por melhor desempenho, a otimização de hiperparâmetros, por busca aleatória ou em grade, é frequentemente utilizada [Bergstra and Bengio 2012].

Para disponibilizar o modelo, utiliza-se uma API REST construída com FastAPI [Ramirez], framework web assíncrono que emprega modelos de dados definidos com Pydantic para validar as requisições.

3. Metodologia

3.1. Ambiente e estrutura do projeto

O projeto foi instanciado em um ambiente virtual Python 3.12.9, com as bibliotecas scikit-learn 1.9.0, MLflow 3.16.0, pandas, numpy e FastAPI. A estrutura de diretórios segue o template FabricaIA, com as pastas config/, data/, models/, notebooks/, src/, tests/ e docs/.

3.2. Dados e pré-processamento

O arquivo data/raw/iris.csv contém 150 linhas e 5 colunas (quatro características numéricas e a coluna alvo target). O pipeline de pré-processamento envolve limpeza, codificação do rótulo e padronização das características, seguido da divisão dos dados em treino e teste (80/20), resultando em 119 amostras de treino.

```
df = load_data(project_path("data", "raw", "iris.csv"))
df_clean = clean_data(df, drop_duplicates=True)
df_encoded = encode_categorical(df_clean) # alvo -> 0/1/2
df_scaled = scale_features(df_encoded) # 4 features (StandardScale)
X_train, X_test, y_train, y_test = split_data(df_scaled, "target")
```

3.3. Treinamento e rastreamento de experimentos

O classificador Random Forest foi configurado com 100 estimadores e profundidade máxima 10, treinado por meio do componente ModelTrainer do projeto. O rastreamento foi realizado com MLflow, no experimento testeprojecto_experiments, com backend SQLite local, registrando parâmetros (n_estimators, max_depth e número de amostras), métricas e o modelo como artefato.

```
model = trainer.train_model(X_train, y_train, "random_forest",
                             run_name="random_forest_baseline", n_estimators=100, max_depth=10)
metrics = trainer.evaluate_model(model, X_test, y_test)
trainer.save_model(model, project_path("models", "trained", "model.pkl"),
                   artifact_path="random_forest_model")
trainer.end_run()
```

3.4. API de predição

Na inicialização, a aplicação FastAPI carrega os modelos de models/trained/*.pkl e deriva o nome do modelo removendo o sufixo _model; como o arquivo persistido é model.pkl, o nome válido é model. O endpoint POST /predict recebe um corpo JSON com as características e o nome do modelo e retorna a classe predita, as probabilidades e a confiança.

```
POST /predict
{ "features": { "sepal_length": 5.1, "sepal_width": 3.5,
               "petal_length": 1.4, "petal_width": 0.2 },
  "model_name": "model" }
```

A resposta retornada é exibida abaixo.

```
{ "prediction": 2,
  "probability": { "0": 0.01, "1": 0.24, "2": 0.75 },
  "confidence": 0.75 }
```

3.5. Validação por testes automatizados

A suíte de testes do projeto foi executada e validou os componentes de processamento, treinamento, engenharia de características e visualização, com 18 testes aprovados. Como o pacote não está instalado de forma editável no ambiente, os testes foram executados com PYTHONPATH apontando para a raiz do projeto.

4. Resultados e Discussão

Os resultados obtidos confirmam a execução do pipeline de ponta a ponta. A Tabela 1 resume as métricas do modelo Random Forest treinado, registradas no experimento teste-projeto_experiments.

Tabela 1. Métricas registradas no MLflow (precisão, recall e F1 para a classe 1).

Run	Acurácia	Precisão	Recall	F1
random_forest_baseline	0,9333	0,90	0,90	0,90
entrega_topicos_avancados	0,9333	0,90	0,90	0,90

O modelo alcançou acurácia de 93,33% e F1 de 0,90, resultado coerente com a simplicidade e a separabilidade do dataset Iris. Como esperado para um modelo baseline, não foi realizada otimização de hiperparâmetros; a escolha de 100 estimadores e profundidade máxima 10 forneceu um desempenho estável. O modelo foi persistido em models/trained/model.pkl e registrado como artefato no experimento teste-projeto_experiments.

Tabela 2. Predições com características padronizadas (0=setosa, 1=versicolor, 2=virginica).

Espécie	Características cruas	Características padronizadas	Predição
setosa	5,1, 3,5, 1,4, 0,2	-1,024, -0,850, -1,368, -1,296	0
versicolor	7,0, 3,2, 4,7, 1,4	0,288, -0,873, 0,228, 0,355	1
virginica	6,3, 3,3, 6,0, 2,5	-0,234, -0,709, 1,013, 1,032	2

A suíte de testes passou integralmente (18 testes aprovados), corroborando a integridade dos componentes.

5. Conclusão

Este relatório documentou a instanciação e a execução do projeto InstanciaIris. Os objetivos foram alcançados: o projeto foi instanciado a partir do template FabricaIA; um pipeline de pré-processamento e treinamento foi construído e executado com sucesso no notebook; o modelo Random Forest foi treinado, avaliado e rastreado em MLflow, com acurácia de 93,33% e F1 de 0,90; o modelo foi persistido e exposto por uma API REST via FastAPI; e o funcionamento foi validado por testes automatizados e por uma chamada de predição real.

Entre as contribuições, destacam-se a definição de um fluxo de trabalho reproduzível, o uso de MLflow para a rastreabilidade de experimentos e a disponibilização de um serviço de inferência.

Referências

- Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.
- Dietterich, T. G. (2000). Ensemble methods in machine learning. In *Multiple Classifier Systems (MCS)*, pages 1–15. Springer.
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188.
- Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1137–1143.
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830.
- Ramirez, S. Fastapi documentation. <https://fastapi.tiangolo.com>. Acessado em 2026.
- Sokolova, M. and Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing and Management*, 45(4):427–437.
- Zaharia, M., Chen, A., Davidson, A., et al. (2018). Accelerating the machine learning lifecycle with mlflow. *IEEE Data Engineering Bulletin*, 41(4):39–45.